

Introduction to Java

Rahul Guha || SME || Java FS Training || HCL

What is Java ?



James Gosling – The mind behind Java 🍷

इसी एक इंसान की वजह से आज लाखों लोग Software Industry में अपना Career बना रहे हैं♥ !

- What is Java?
- Java is a **High-level, object-oriented programming language**.
 - **High-level:** A language that is easy for humans to read, write, and understand (close to English).
 - **Object-oriented:** A way of programming where everything is designed using real-world objects like Student, Car, or Bank.
 - **Programming language:** A language used to give instructions to a computer to perform tasks.

What is Java ?

- Types of Java Editions
- There are total 3 editions in Java which are as below :-
- **1. Java Standard Edition (Java SE)**
 - *Core Java platform* used to build **desktop, console, and basic applications**.
 - Includes **JVM, core libraries**, and *fundamental APIs*.
- **2. Jakarta Enterprise Edition (Jakarta EE) (formerly Java EE)**
 - Used for *web applications* and **enterprise-level applications**.
 - Supports technologies like **Servlets, JSP, REST APIs** and *Microservices*.
- **3. Java Micro Edition (Java ME)**
 - Used for **embedded systems** and *resource-constrained devices*.
 - Common in **IoT**, *smart cards* and *small devices*.

History of Java

Below is the basic history of Java :-

Year	Version / Event	Key Highlights
1991	Java Development Started	<i>Green Project</i> at Sun Microsystems
1995	Java Announced	Public launch, <i>beta versions</i>
1996	Java 1.0	First official release
2004	Java 5	Generics, Annotations, Enhanced for-loop
2010	Oracle Acquired Sun	Java ownership moved to <i>Oracle</i>
2014	Java 8	Lambda Expressions, Stream API
2017	Java 9	Module System (<i>Project Jigsaw</i>)
2018	Java 11 (<i>LTS Version</i>)	HTTP Client API, var in lambdas
2021	Java 17 (<i>LTS Version</i>)	Sealed Classes, Strong Encapsulation
2023	Java 21 (<i>LTS Version</i>)	Virtual Threads, Record Patterns
2025	Java 25 (<i>LTS Version</i>)	Flexible Constructors, Module Imports

Use of Java

Below are some uses of Java :-

- **Enterprise Applications**

- Java is widely used in *large-scale enterprise systems*.
- Used in *banking, finance, ERP, CRM and inventory management systems*.
- Preferred because of its *stability, scalability and reliability*.

- **Web Applications**

- Java is extensively used for *server-side web development*.
- Technologies include *Servlets, JSP, Spring and Spring Boot*.
- Commonly used to build *secure and scalable web applications*.

- **Backend & Server-side Applications**

- Java is a popular choice for building *REST APIs and microservices*.
- Used in *distributed systems* and *high-traffic backend services*.
- Common in *enterprise* and *cloud-based architectures*.

- **Android Mobile Applications**

- Java is used to develop *Android applications* using the Android SDK.
- Many existing Android apps are built using *Java*.
- Still widely supported, though newer apps may also use *Kotlin*.

- **Cloud-based Applications**

- Java is used to build *backend services* deployed on cloud platforms.
- Commonly deployed on *AWS, Google Cloud and Microsoft Azure*.
- Used for *scalable* and *distributed cloud systems*.

Use of Java

Below are some uses of Java :-

- **Big Data Technologies**

- Java is heavily used in *big data frameworks*.
- Technologies like *Hadoop* are written in Java.
- Used for *large-scale data processing*.

- **Desktop Applications**

- Java can be used to create *desktop software*.
- Technologies include *JavaFX* and *Swing*.
- Usage is *moderate* compared to web and enterprise applications.

- **Software Development Tools**

- Many popular *IDEs and development tools* are built using Java.
- Examples include *Eclipse, NetBeans and IntelliJ IDEA*.
- Mostly relevant to *developers* and development environments.

- **Embedded Systems & IoT**

- Java ME is used in *embedded devices* and *IoT systems*.
- Applied in *smart cards, sensors and small devices*.
- Usage is *limited* compared to enterprise applications.

- **Scientific & Research Applications**

- Java is used in *simulations, research tools and scientific computing*.
- Chosen for its *reliability* and *portability*.
- Limited to *specific academic and research use cases*.

- **Game Development**

- Java is mainly used for *simple, educational and learning-based games*.
- Technologies include *JavaFX* and *Swing*.
- Rarely used for *high-end commercial game development*.

Features of java

- **Platform Independent**

- → Java is platform independent because a Java program can be compiled on one machine and executed on another machine.
- → Java source code is compiled into bytecode, which is not dependent on any operating system.
- → JVM is platform dependent, but it allows the same bytecode to run on different machines.

- **Object Oriented**

- → Java is an object-oriented programming language.
- → It uses concepts like class and object to organize code.
- → Features like encapsulation, inheritance, polymorphism, and abstraction make programs reusable and easy to maintain.

- **Simple**

- → Java syntax is simple and easy to understand.
- → It removes complex features like pointers and operator overloading.

- → Automatic memory management reduces the burden on developers.

- **Secure**

- → Java provides a secure runtime environment.
- → It does not allow direct access to memory using pointers.
- → JVM, class loader, and bytecode verifier protect applications from malicious code.

Features of java

- **Robust**

- → Java is designed to be reliable and error-resistant.
- → It provides strong exception handling mechanisms.
- → Automatic garbage collection helps prevent memory leaks.

- **Multithreaded**

- → Java supports multithreading, allowing multiple tasks to run at the same time.
- → Each thread executes independently within a program.
- → Multithreading improves application performance and responsiveness.

- **High Performance**

- → Java provides good performance using JVM and JIT compiler.
- → Frequently executed code is converted into native machine code.
- → This reduces execution time and improves speed.

- **Distributed**

- → Java supports the development of distributed applications.
- → Applications can communicate over a network using Java APIs.
- → Java is widely used in web services and cloud-based systems.

Features of java

- **Portable**

- → Java programs are portable due to platform-independent bytecode.
- → The same compiled code runs on different operating systems.
- → No recompilation is required for different platforms.

- **Dynamic**

- → Java supports dynamic behavior at runtime.
- → Classes can be loaded, linked, and modified during execution.
- → This makes Java applications flexible and extensible.

- **Rich API**

- → Java provides a large collection of built-in libraries.
- → APIs support database connectivity, networking, and GUI development.
- → These libraries reduce development time and effort.

- **Scalability**

- → Java applications can scale from small to large systems.
- → It supports multithreading and distributed computing.
- → Java is widely used in enterprise and cloud applications.

Features of java

- **Versatile**

- → Java can be used in many different types of applications.
- → It is used in web applications, mobile apps, and enterprise software.
- → Java suits both small and large projects.

- **Community Support**

- → Java has a large and active developer community.
- → Many frameworks, tools, and libraries are available.
- → Learning resources and support are easily accessible.

Integrated Development Environments (IDEs)

- **Introduction :** IDEs are advanced software tools to write Java Code including development, debugging, project management etc.
- **Examples :**
 - IntelliJ IDEA
 - Eclipse
 - NetBeans
 - VS Code
 - etc...

What is JDK?

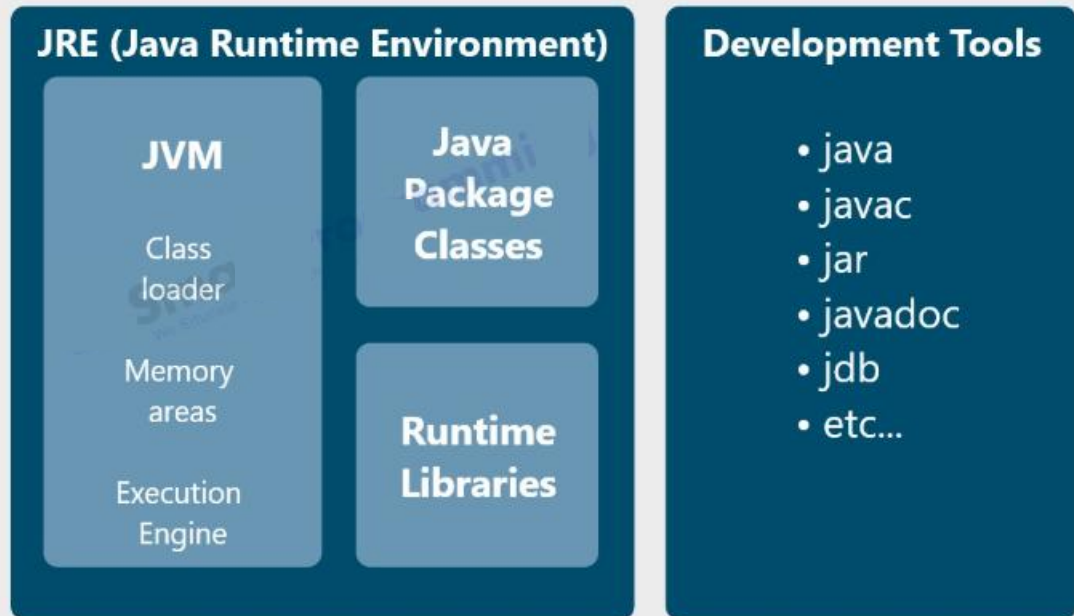
- The Java Development Kit (JDK) is a software development kit used to develop applications in the Java programming language.
- It includes
 - various tools which are essential for compiling, running, documenting, packaging, and debugging Java applications, such as The Java Application Launcher (java), Java Compiler (javac), Documentation Generator (javadoc) etc and
 - a runtime environment (Java Runtime Environment or JRE).

**JDK = DEVELOPMENT TOOLS + RUNTIME ENVIRONMENT
OR JRE
DEVELOP +
TEST +
DEPLOY +
PROPERTY CHECK**

What is JDK?

JVM -> JRE -> JDK

(JDK) Java Development kit



What is JDK ?

- JDK stands for “Java Development Kit”
- JDK, as name suggests, is a “Software kit (or package)” which contains:-
 - Development Tools ←
 - Runtime Environment ←
- JDK is used to develop, test and deploy Java applications

Development Tools

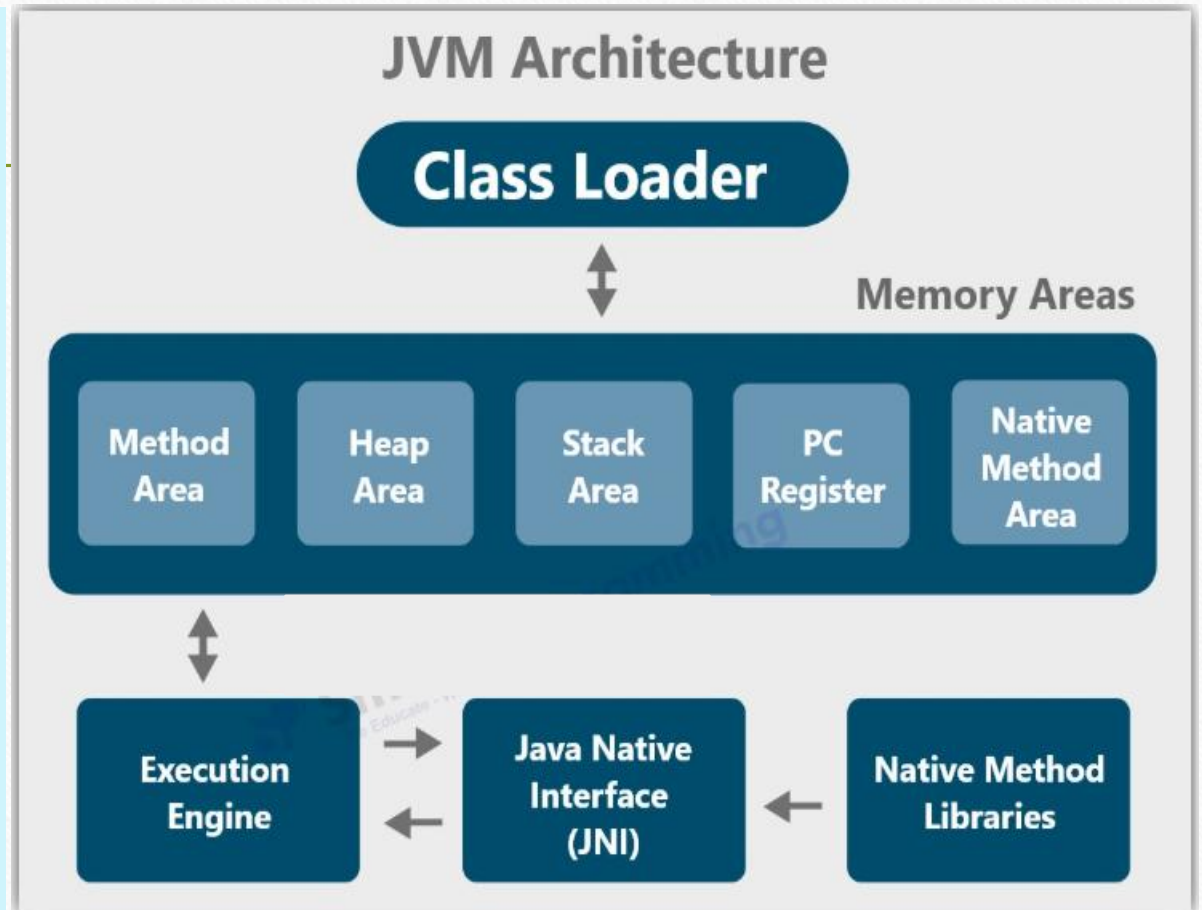
Tool	Description
java	The Java Application Launcher: Executes Java applications and applets. It replaces the older jre tool, combining development and deployment functionalities.
javac	Java Compiler: Converts Java source code into Java bytecode, producing .class files.
javadoc	Documentation Generator: Automatically creates HTML documentation from Java source code comments.
jar	Archiver and Manager: Packages related class libraries into a single JAR (Java ARchive) file and manages JAR files.
jdb	Java Debugger: Provides a command-line interface for debugging Java programs.

Other Development Tools

Tool	Description
appletviewer	Runs and debugs Java applets without a web browser.
javah	C Header and Stub Generator: Generates C header files and JNI (Java Native Interface) stubs for native methods.
javap	Class File Disassembler: Displays information about Java class files, including bytecode and structure.
javaws	Java Web Start Launcher: Launches and manages Java applications distributed via JNLP (Java Network Launch Protocol).
jps	Java Virtual Machine Process Status Tool: Lists HotSpot JVMs on the system and provides process details.
jrunscript	Command-Line Script Shell: Allows execution of JavaScript and other scripting languages from the command line.
jstack	Prints Java stack traces of Java threads, useful for diagnosing thread-related issues (experimental).
jstat	JVM Statistics Monitoring Tool: Monitors various JVM statistics, such as garbage collection and memory usage (experimental).
jstatd	jstat Daemon: Provides a network interface to monitor JVM statistics (experimental).

What is JVM ?

- The JVM is a part of the Java platform that executes Java bytecode, converting it into machine code for the host system. It provides an abstraction layer between compiled Java code and the operating system.
- The main use of the JVM is to enable platform-independent execution of Java programs by interpreting or compiling bytecode to machine code, managing memory, ensuring security, and optimizing performance.
- Below is the diagram showing JVM architecture in JRE.



What is JVM ?

JVM (Java Virtual Machine) =

Class Loader + Memory Areas + Execution Engine

What is JVM ?

JVM = THAT DOES NOT EXIST PHYSICALLY

- JVM stands for “Java Virtual Machine”
- JVM, as name suggests, is a “Virtual Machine” which
 - takes the byte code (from .class file)
 - translates into machine specific code
 - and run on any platform
- JVM is “Platform Dependent” which makes “Java Platform Independent”



The JVM is like Cricket ground where the players perform, same JVM is Virtual Machine where Java bytecode is executed.

Java Virtual Machine (JVM)

- Byte Code Verifier
- Memory Areas
- Execution Engine

- There are 3 main components in JVM :-
- **Class Loader:** The class loader is responsible for dynamically loading Java classes into memory at runtime. It handles loading, linking, and initialization, organizing classes into namespaces and ensuring they are available for execution.
- **Memory Areas:** The JVM's memory areas include the heap (for objects), stack (for method calls and local variables), method area (for class-level data), and native method stacks. These regions manage memory allocation, storage of program data, and execution context.
- **Execution Engine:** The execution engine interprets or compiles bytecode into machine code for execution by the CPU. It includes the interpreter for line-by-line execution and the Just-In-Time (JIT) compiler for optimizing frequently executed code paths.

What is JRE ?

- The Java Runtime Environment (JRE) is a software platform that enables the execution of Java applications. It works by providing a runtime environment that includes a set of core libraries and Java Virtual Machine (JVM).
- The JRE allows Java programs to run on any device or operating system without modification, by translating Java bytecode into native machine code that the JVM executes.
- Below is the diagram showing JRE architecture in JDK.

What is JRE ?

What is JRE ?

- JRE stands for “Java Runtime Environment”
- JRE, as name suggests, provides a **runtime environment** that helps to run Java programs
- It doesn't contain tools for developing Java programs, thus it's suitable for end users who just want to run Java applications.

- **Java Runtime Environment (JRE):**

- **Function:** Provides the libraries, Java Virtual Machine (JVM), and other components to run Java applications.
- **Included:** The JRE is part of the JDK, and it includes the JVM along with the necessary runtime libraries.

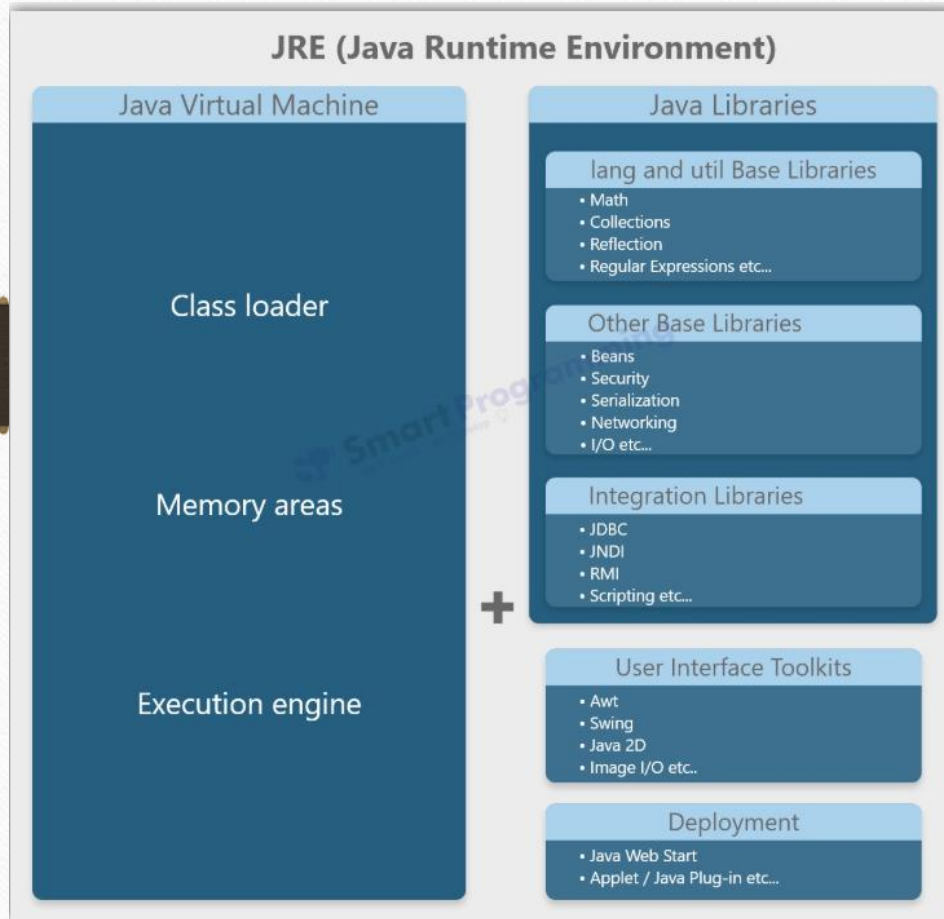


The Java Runtime Environment (JRE) is like the stadium in cricket. Just as a stadium is where the game is played, the JRE provides the environment to run Java applications, with the necessary libraries & resources, but without the development tools.

**JRE =
JAVA PACKAGE CLASSES
+
RUNTIME LIBRARIES +
JVM**

GO TO LIB>SRC>BASE FOLDER

What is JRE ?



**JRE (Java Runtime Environment) = Java Libraries + Unser Interface
Toolkits + Deployment + JVM (Java Virtual Machine)**

Usage of JRE :-

Java Runtime Environment (JRE) is used to provide a runtime platform for executing Java applications. It includes the JVM, core libraries, and resources required to run Java programs, ensuring platform independence and handling tasks like memory management, security, and execution of compiled Java code.

Components of JRE

- **Java Libraries**

- Java libraries are collections of pre-written code that provide essential functionality for development. They include base libraries (e.g., `java.lang`, `java.util`), additional base libraries (e.g., `java.beans`, `java.security`), and integration libraries (e.g., `JDBC`, `JNDI`) for external system integration.
- These are divided in 3 main categories :-
 - **lang & util Base Libraries (e.g., `java.lang`, `java.util`):** Provide core classes for Java, including fundamental utilities (e.g., collections, strings, math) and language features (e.g., threading, exceptions).
 - **Other Base Libraries (e.g., `java.beans`, `java.security`):** Provide additional functionality, such as object introspection (beans) and security features (cryptography, authentication).
 - **Integration Libraries (e.g., `JDBC`, `JNDI`):** Offer APIs for database connectivity (`JDBC`), directory services (`JNDI`), and integrating with external systems like databases and networks.

Components of JRE

- **User interface toolkits :-** User interface toolkits in JRE, like AWT, Swing, and Java 2D, provide tools for building graphical user interfaces (GUIs). They offer components for windows, buttons, and graphics rendering, enabling developers to create interactive, visually-rich applications with support for both basic and advanced UI elements and 2D graphics.
- **Deployment :-** Deployment in JRE involves packaging and distributing Java applications for execution on client machines. This includes delivering Java programs in formats like JAR or WAR files, ensuring they run within the Java Runtime Environment, often using tools like Java Web Start or applet technology for deployment.
- **Java Virtual Machine :-** The Java Virtual Machine (JVM) is an engine that executes Java bytecode. It provides platform independence by allowing Java programs to run on any device with a JVM, handling tasks like memory management, garbage collection, and code optimization.

Components of JRE

- **User interface toolkits :-** User interface toolkits in JRE, like AWT, Swing, and Java 2D, provide tools for building graphical user interfaces (GUIs). They offer components for windows, buttons, and graphics rendering, enabling developers to create interactive, visually-rich applications with support for both basic and advanced UI elements and 2D graphics.
- **Deployment :-** Deployment in JRE involves packaging and distributing Java applications for execution on client machines. This includes delivering Java programs in formats like JAR or WAR files, ensuring they run within the Java Runtime Environment, often using tools like Java Web Start or applet technology for deployment.
- **Java Virtual Machine :-** The Java Virtual Machine (JVM) is an engine that executes Java bytecode. It provides platform independence by allowing Java programs to run on any device with a JVM, handling tasks like memory management, garbage collection, and code optimization.

Overview

- **JDK – Dev Tools + Runtime Env.**
Used for **Development , Testing, Deploy,**
- **JVM – Class Loader + Method Area + Exception Engine**

Method Area – Class Level Data

Heap - For Object

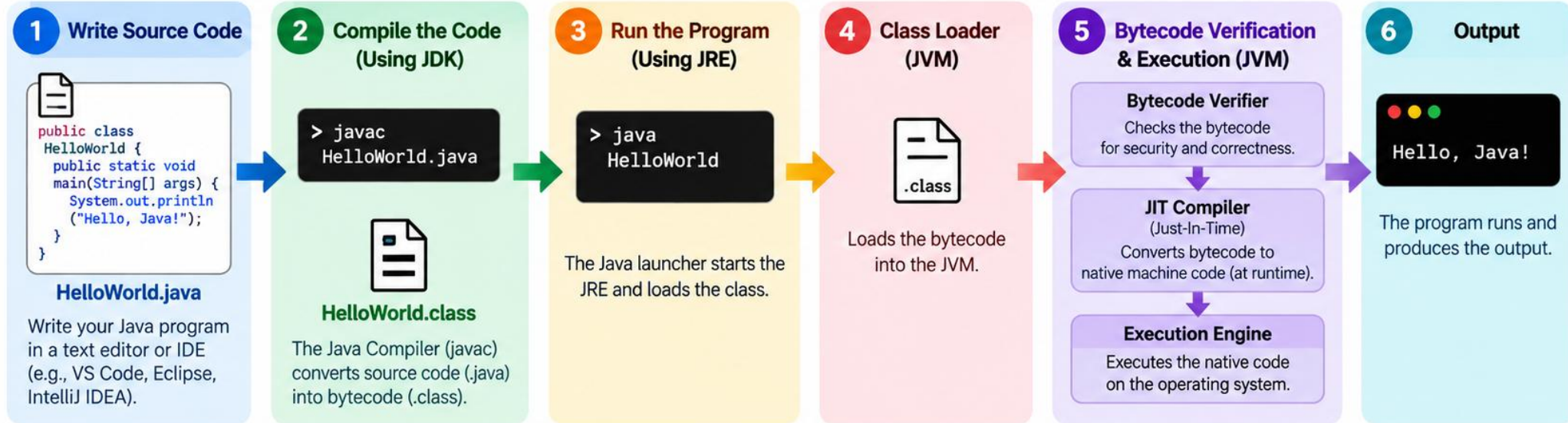
Stack- For Method Calls

PC Reg + Native Methods

- **JRE**
- **Java Libraries + Interfaces + Toolkit + Deployment + JVM**

Java Execution Steps

From Source Code to Output



Key Components



JDK (Java Development Kit)

- Includes JRE + development tools
- Provides javac (compiler), java (runtime), debugger, etc.
- Used for development.



JRE (Java Runtime Environment)

- Includes JVM + core libraries
- Provides the runtime environment
- Used to run Java programs.



JVM (Java Virtual Machine)

- Executes the bytecode
- Provides platform independence
- Includes class loader, bytecode verifier, JIT compiler, and execution engine.



Java Class Libraries

- Pre-built classes (e.g., java.lang, java.util)
- Provide useful functionality
- Included in both JDK and JRE.

Write Once, Run Anywhere

Java converts your code into platform-independent bytecode, which runs on any system with a JVM.

Structure of Java Program

- When writing a program in any programming language, it's important to follow a basic structure recommended by experts. Typically, a Java program is made up of the following parts, as shown in the figure.



Structure of Java Program

- **1. Package Declaration Statement (Optional)**

- The package declaration specifies the package name for the Java class. It is the first statement in a Java program (if present).

- **Purpose :** Package statement is used to group related classes and organize the code.

Syntax : `package packageName;`

Example : `package com.example.myapp;`

- **Import Statements (If Necessary)**

- The import statement is used to include other Java classes or packages in the program, enabling the use of pre-defined classes (e.g., `java.util.Scanner`).
- **Purpose :** Import statement is used to import built-in or user-defined classes for reusability.

Syntax :

- `import packageName.ClassName; // Imports a specific class`
- `import packageName.*; // Imports all classes in the package`

Example :

- `import java.util.Scanner; // Imports Scanner class`
- `import java.util.*;`

Structure of Java Program

- **Class Definition Statements**

- The class is the fundamental building block of a Java program. Every Java program must have at least one class.
Syntax :

- ```
class ClassName {
 // Class body
}
```

Example :

- ```
public class Car {  
    // Class members go here  
}
```

- **We can create any number of classes in one program.**

- There should be atleast one main class which should contains the main() method as main() method is the entry point for the program.

- A class can include the following components which are known as "Class Members":

- **Variables** : Also called fields or attributes, which hold data.
- **Constructors** : Used to initialize objects.
- **Methods** : Define behaviors or actions of the class.
- etc...

Structure of Java Program

- **Variables Declaration Statements (Optional) [Class Members]**

- Variables are used to store data and are declared within the class (as fields) or within methods (local variables).

Syntax :

- `dataType variableName = value; //`
Field or local variable declaration

Example :

- `String model = "Tata Nexon" //`
Field
- `int year = 2020; //` Field

- **Constructors Declaration Statements (Optional) [Class Members]**

- Constructor is a special method having same name as that of class name but should not have any return type.
- A constructor initializes objects of a class. If no constructor is provided, Java supplies a default constructor.

Syntax :

- `ClassName() {`
- `// Constructor body`
- `}`

Example :

- `public Car() {`
- `System.out.println("Constructor called!");`
- `}`

Structure of Java Program

- **Methods Declaration Statements [Class Members]**

- Methods are the set of codes which performs a particular task.
- We can create any number of methods in one class.

- **Syntax :**

- `accessModifier returnType methodName(parameters) {`
- `// Method body`
- `}`

- **Example :**

- `public void start() {`
- `System.out.println("Car Started");`
- `}`

- **Main Method [Class Members]**

- `main()` method is the entry point for java program from where program execution starts.
- `main()` method can have different syntax but below is the standard syntax that we should use.

- **Syntax :**

- `public static void main(String[] args) {`
- `// Code to execute`
- `}`

- **Example :**

- `public static void main(String[] args) {`
- `System.out.println("Hello Deepak...!!");`
- `}`

Structure of Java Program

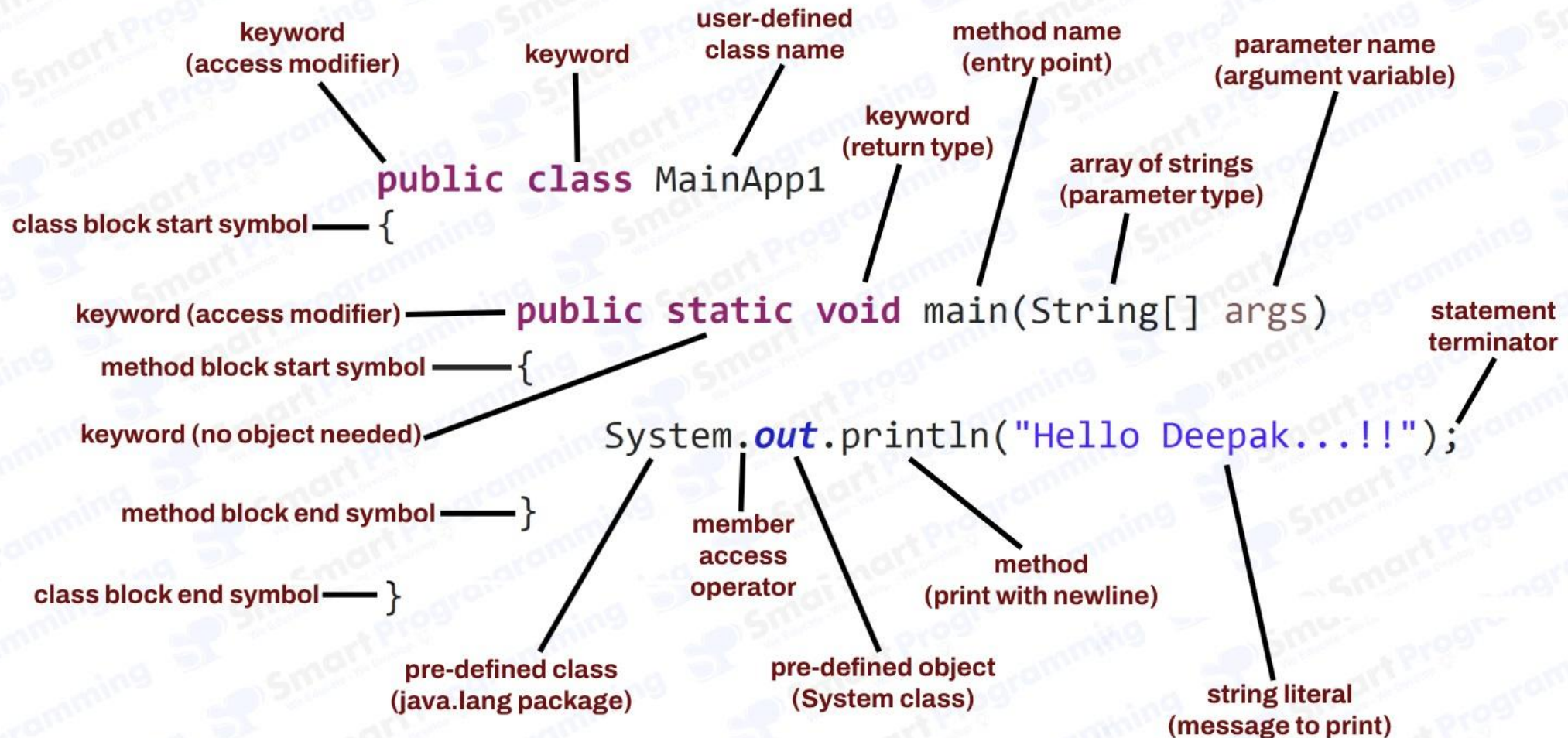
- **Simple Java Hello Program**

- In last tutorial, we have deeply explained the Structure of Java Hello Program. Now, below is the simple Java Hello Program.

MainApp.java

- `public class MainApp`
- `{`
- `public static void main(String[] args)`
- `{`
- `System.out.println("Hello Deepak");`
- `}`
- `}`

Structure of Java Program



Structure of Java Program

- 1. public (Keyword - Access Modifier)
 - The "public access modifier keyword" declares that the MainApp class is accessible from anywhere in the program (other packages are also included) and JVM can invoke it from outside the class.
 - **Note** that when a class is declared as public, it must have the same name as the file name (in this case, MainApp.java).
- 2. class (Keyword)
 - The "class keyword" is used to define a class in Java, which acts as a blueprint for creating objects.
 - The class can contain :-
 - **Variables:** Store data or attributes of the class.
 - **Constructors:** Initialize the object's state.
 - **Methods:** Define the behavior or actions of the class.

Structure of Java Program

- **3. MainApp (User-Defined Class Name)**
 - The "MainApp" is the user-defined class name which contains the overall program.
 - The class must follow the Java naming conventions (CamelCase starting with an uppercase letter).
 - The "MainApp" class acts as the entry point of the program because it contains the main method.
- **4. { (Class Opening Curly Brace)**
 - Here "{" denotes the beginning of the class definition.
 - Every class body is enclosed in curly braces.
- **5. public (Keyword - Access Modifier)**
 - The "public access modifier keyword" here makes the main method accessible to the Java Virtual Machine (JVM) so it can execute the program.
- **6. static (Keyword)**
 - The "static keyword" denotes that the main method belongs to the class rather than an instance of the class.
 - It means that the main method is called without creating an object of the MainApp class.

Structure of Java Program

7. void (Keyword - Return Type)

The "void return-type keyword" indicates that the main method does not return any value.

Since the program's execution does not require any return value from the main method, it is declared void.

8. main (Pre-Defined Method)

The "main pre-defined method" is an entry point of the program from where execution begins.

The JVM looks for this specific method (main() method) to start the program. If there is no main() method then JVM will not start the program execution.

9. String[] (Parameter Type - Array of Strings)

The "String[] parameter type" is used for Command-line Arguments. It holds the arguments passed to the program when it is executed via the command line.

For example: If we run `java MainApp Hello Deepak`, the array will contain:

```
String[0] = "Hello"
```

```
String[1] = "Deepak"
```

Structure of Java Program

10. args (Parameter Name - Argument Variable)

The "args" is the user-defined variable name for the String[] parameter.

"args" is the short name for arguments, but can be renamed to any valid identifier (e.g., data or parameters).

It is used to hold the command-line arguments passed to the program.

11. { (Method Opening Curly Brace)

Here "{" denotes the beginning of the main method body.

12. System (Pre-Defined Class Name)

"System" is the built-in Java class from java.lang package.

It provides access to system-level functionality, such as input/output streams, properties, and environment variables.

It contains:

in: Standard input stream (e.g., keyboard input).

out: Standard output stream (e.g., console output).

err: Standard error stream (e.g., error messages).

Structure of Java Program

13. . (Member Access Operator)

"." is the member access operator which is used to access fields, methods and nested classes of a class or an object.

For example: `System.out` accesses the `out` field of the `System` class.

It contains:

`in`: Standard input stream (e.g., keyboard input).

`out`: Standard output stream (e.g., console output).

`err`: Standard error stream (e.g., error messages).

14. `out` (Pre-Defined Object)

"`out`" is the static member (field or property) of "`System`" class that represents the standard output stream (usually the console).

It is used to print the data like strings, numbers or objects to the console.

15. `println` (Pre-Defined Method)

"`println`" is the pre-defined method of "`PrintStream`" class (referenced by `System.out`).

It prints the given text or data to the console and moves the cursor to a new line.

Structure of Java Program

- **16. "Hello Deepak" (String Literal)**
 - "Hello Deepak" is the sequence of characters enclosed in double quotes.
 - The text Hello Deepak is passed as an argument to the println method and will be displayed in the console.
- **16. ; [Semicolon] (Statement Terminator)**
 - ";" marks the end of a statement in Java. Every executable statement must be terminated with a semicolon.
- **17. } (Method Closing Curly Brace:)**
 - Here "}" marks the end of the main method body.
- **18. } (Class Closing Curly Brace)**
 - Here "}" marks the end of the class definition.

Primitive vs Non-Primitive

- 1. Primitive Data Types

- Primitive data types are the **basic, built-in data types** provided by Java. They store the **actual value** directly.

```
int age = 25; double salary = 45000.50; char grade = 'A'; boolean status = true;
```

- 2. Non-Primitive Data Types

- Non-primitive data types are also called **reference types**. They refer to objects rather than directly storing the object itself.
String name = "Rahul";

- int[] marks = {80, 90, 85};
String || Arrays || Classes || Objects
|| Interfaces || Enums

Feature	Primitive	Non-Primitive
Meaning	Basic data type	Reference type
Provided by	Java language	Java libraries / programmer
Number	8	No fixed limit
Stores	Actual value	Reference to an object
Can call methods?	✗ No	☑ Yes
Can be null?	✗ No	☑ Yes
Examples	int, char, double	String, Array, Class
Default value for instance fields	0, false, '\u0000', etc.	null
Memory behavior	Holds primitive value	Reference points to an object

Identifiers in Java

- An identifier is a name used to identify a program element such as a class, variable, method, or parameter.

Java Identifier

- |
- |—— Cannot start with digit ✗
- |—— Cannot contain spaces ✗
- |—— Cannot be a keyword ✗
- |—— Cannot use special symbols like @, #, - ✗
- |
- |—— Letters + digits + _ + \$ ✓